TD3
Couverture de code, couverture de test

Thomas Sarno

Septembre 2026

1 Introduction

Il vous est demandé de rédiger un compte-rendu pour ce TD dans lequel vous ferez figurer toutes les remarques pertinentes qui vous semblent nécessaires à la bonne compréhension du problème posé et de la solution apportée. Bien évidemment il s'agit d'un travail personnel.
Dans le TD3 nous allons nous concentrer sur la couverture de code de notre AES. L'objectif est de valider que toutes les fonctionnalités de l'AES.

Ci-dessous, pour rappel, vous trouverez un diagramme bloc (Figure 1) ainsi qu'un chronogramme (Figure 2) décrivant le fonctionnement de cet AES.

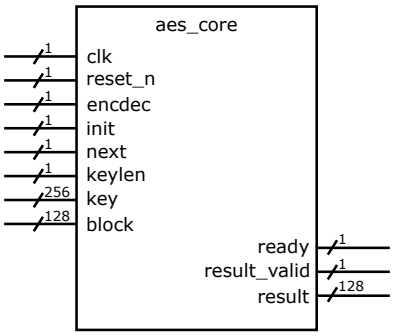


FIGURE 1 – aes_core

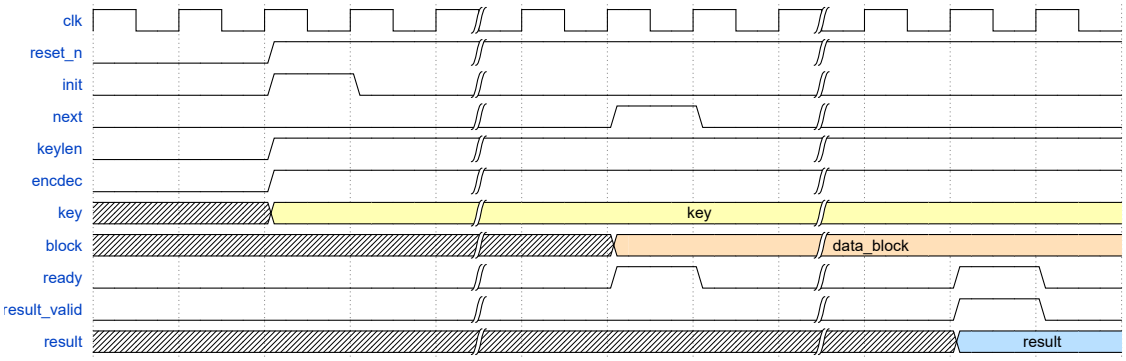


FIGURE 2 – Chronogramme de l'aes_core

2 Couverture de code

2.1 Couverture basique

Maintenant que la vérification fonctionnelle est terminée, il faut analyser un peu plus en détail le hardware testé. La couverture de code nécessite une seconde compilation avec les options idoines.

Modifiez le script `build.sh` en ajoutant un argument `-v` qui ajoute la commande :
`vopt aes_tb -o aes_tb_cov -work work +cover=sbcfte`.
Cette commande permet de générer un objet intermédiaire à simuler à la place du `aes_tb` simulé habituellement.

Puis simulez comme habituellement mais au lieu de simuler le modèle `aes_tb`, remplacez-le par `aes_tb_cov` : pour la couverture, il faut simuler le modèle compilé avec les options de coverage. Il faut également ajouter l'option de compilation `-coverage`.
Vous pouvez activer et désactiver l'option `-coverage` avec l'argument `-v` défini plus haut pour vous faciliter la tâche.

Exécutez la simulation pour connaître la couverture de l'AES. Attention, la couverture inclut le testbench, dans le cadre de la vérification de l'AES, les parties non couvertes du testbench ne sont pas un problème. Il sera peut-être nécessaire d'augmenter le nombre d'itérations pour arriver à la couverture de code maximale.

Certaines parties du code ne sont pas complètement couvertes. Voyez si c'est dû à un mauvais test ou à des lignes inaccessibles en fonctionnement normal.
Vous aurez noté que le design que nous étudions permet de faire des chiffrements et des déchiffrements avec des tailles de clés de 128 bits ou de 256 bits.
Il va donc falloir tester toutes ces fonctionnalités sans oublier de faire des vérifications fonctionnelles.

2.2 Augmentation de la couverture

Pour pouvoir atteindre un pourcentage de couverture plus élevé, il faut simuler toutes les fonctionnalités du circuit testé. Pour le moment, seule la fonctionnalité de chiffrement 128 bits est testée, il va falloir mettre en place les tests des autres fonctionnalités.
Pour prendre en compte les différents paramètres nécessaires à ces tests, la task `setup_operation()` doit être déportée dans la classe `Inputs` pour pouvoir plus simplement prendre en compte les tirages aléatoires de paramètres.
Il faut aussi ajouter des contraintes sur les tirages pour les paramètres *enc_dec* et *key_len* pour limiter leur portée à {0,1}. Vous pouvez utiliser les paramètres *AES_128_BIT_KEY*, *AES_256_BIT_KEY*, *AES_DECIPHER* et *AES_ENCIPHER* en les déplaçant également dans `Inputs`.
Il faut ensuite étendre la fonction python qui calcule les références pour qu'elle prenne en compte le déchiffrement. On rappelle pour cela que la méthode `AES.decrypt` permet de faire un déchiffrement avec, de façon similaire au chiffrement avec `AES.encrypt`.
Et pour finir, il faut faire en sorte que l'appel de cette fonction depuis le testbench soit faîte avec les bons arguments.